

Web-Based Extraction Tool

Semester Project

Student Name: HAMMAD AHMAD

SAP ID: 63420

Course: Operating Systems

Semester: 4

1. Project Overview

The Web-Based Extraction Tool is a robust Python application designed to automate the process of gathering information from across the web. It bridges the gap between raw web content and structured analysis, providing a user-friendly interface to scrape, process, and export information without requiring programming expertise.

2. Problem Statement

Web content is primarily locked in HTML format, making it difficult for researchers and businesses to analyze efficiently. Manual collection is prone to errors and time-consuming. This project provides a simplified, yet powerful alternative for automated information retrieval.

3. Core Capabilities

A. Web Scraping Engine

- URL-Based Extraction: Fetches content from any publicly accessible URL.
- Targeted Selectors: Uses CSS selectors for precise data targeting.
- Attribute Mapping: Extracts links, images, and specific HTML metadata.

B. Intelligent Navigation

- Auto-Pagination: Detects and follows "Next" buttons and page numbers.
- Domain Safety: Validates URLs to stay within the target website domain.

C. Asynchronous Architecture

- Background Processing: Long-running tasks are handled via Celery queues.
- Task Isolation: Web interface remains responsive during heavy scraping jobs.
- Status Monitoring: Real-time progress tracking for the user.

D. Export & Clean-up

- Automated Cleaning: Removes duplicates and normalizes text content.
- Standard Formats: Exports findings into clean CSV files for external analysis.

4. T

Frontend Interface

HTML5, CSS3, and JavaScript with AJAX for seamless status updates.

Backend Logic

Python/Flask (Framework), Celery (Task Queue), and Redis (Message Broker).

Extraction Engine

BeautifulSoup4 for parsing, Requests for HTTP, and Pandas for processing.

5. OS Principles & Implementation

- Concurrency: Leveraging Celery workers for parallel task execution.
- IPC: Redis serves as the communication bridge between separate processes.
- System Resources: Implemented rate-limiting to manage network I/O.

6. S

-
- Politeness Policy: Small delays prevent server overload on target sites.
 - Public Access: Only scrapes data that is publicly available.
 - Input Sanitization: Protects the system against malicious URL injections.

7. F

-
- Database Storage: Permanent persistence of results in SQL/NoSQL.
 - Dynamic Rendering: Support for JavaScript-heavy sites via Playwright.
 - Visual UI: Point-and-click interface for element selection.